



杭州电子科技大学
HANGZHOU DIANZI UNIVERSITY

《数据结构课程设计》

实验报告

实验 4：查找

姓 名： xx

学 号： xx

专 业： 网络空间安全

实验时间： 2024.12.19-2024.12.26

指导老师： xx

杭州电子科技大学
网络空间安全学院

一、实验目的

- 1. 掌握静态查找表的构造与查找算法的实现；
- 2. 掌握动态查找表（BST）的构造与查找、插入、删除算法的实现；
- 3. 掌握哈希表的构造与查找算法的实现。

二、实验内容与实验结果

实验 4-1 静态查找的实现

1. 实验内容

I. 静态查找表基本操作的实现

根据定义的存储结构，编写下列基本操作函数的 C/C++源代码：

（1）编写一个创建静态查找表的函数 `CreateSSTable(SSTable &ST, int n)`，其中 `n` 为元素个数。创建的静态查找表从下标 1 开始存放数据(0 元素可以用作“哨兵”)。

注：为提高程序调试速度，该函数允许直接用赋值语句将各数据元素的值以“常量”形式赋给静态查找表的各个数组元素。

（2）编写一个遍历函数 `TraversSSTable(SSTable ST)`，输出静态查找表 `ST` 的所有数据元素。

（3）编写一个顺序查找函数 `int SearchSSTable_Seq(SSTable ST, KeyType key, int &c)`，其中参数 `key` 为要查找的关键字，参数 `c` 保存查找过程中关键字比较的次数。若查找成功，函数返回该关键字元素所在的下标，否则，查找不成功，返回 0（有“哨兵”）或-1（无“哨兵”）。

（4）编写一个排序函数 `SortSSTable(SSTable &ST)`，将静态查找表所有数据元素按关键字大小递增排序（有序表）。

（5）编写一个折半查找函数 `int SearchSSTable_Bin(SSTable ST, KeyType key, int &c)`，其中参数 `key` 为要查找的关键字，参数 `c` 保存查找过程中关键字比较的次数。若查找成功，函数返回该关键字元素所在的下标，否则，查找不成功，返回-1。

II. 编写一个主函数 `main()`，检验上述操作函数是否正确，实现以下操作：

（1）定义一个静态查找表 `ST1`，将下列各元素的值写入 `ST1`，然后输出 `ST1` 的各元素的值。

学号 key	姓名 name	性别 sex	年龄 age
56	Zhang	F	19
19	Wang	F	20
80	Zhou	F	19
5	Huang	M	20
21	Zheng	M	20

64	Li	M	19
88	Liu	F	18
13	Qian	F	19
37	Sun	M	20
75	Zhao	M	20
92	Chen	M	20

(2) 编写一段代码，根据输入的一个关键字，顺序查找静态表中的数据元素，若查找成功，输出找到的元素（记录）的值和关键字的比较次数，否则输出“当前要查找的记录不存在！”和关键字的比较次数（查找不成功的 ASL）。该段代码可以重复上述操作多次，关键字输入“-1”时结束。

(3) 依次使用静态查找表中的关键字进行顺序查找，计算并输出查找成功的 ASL。

(4) 将静态查找表 ST1 按关键字递增排序，变成一个有序表，然后输出有序表的各元素的值。

(5) 编写一段代码，根据输入的一个关键字，折半查找静态表中的数据元素，若查找成功，输出找到的元素（记录）的值和关键字的比较次数，否则输出“当前要查找的记录不存在！”和关键字的比较次数。该段代码可以重复上述操作多次，关键字输入“-1”时结束。

(6) 依次使用静态查找表中的关键字进行折半查找，计算并输出查找成功的 ASL。

(7) 选择使用不在静态查找表中的关键字进行折半查找，计算并输出查找不成功的 ASL。

2. 源程序代码 (*.CPP)

（见附件 static_search.cpp）

3. 运行结果

```

学号无序的静态查找表
学号：56， 姓名：Zhang， 性别：F， 年龄：19
学号：19， 姓名：Wang， 性别：F， 年龄：20
学号：80， 姓名：Zhou， 性别：F， 年龄：19
学号：5，  姓名：Huang， 性别：M， 年龄：20
学号：21， 姓名：Zheng， 性别：M， 年龄：20
学号：64， 姓名：Li， 性别：M， 年龄：19
学号：88， 姓名：Liu， 性别：F， 年龄：18
学号：13， 姓名：Qian， 性别：F， 年龄：19
学号：37， 姓名：Sun， 性别：M， 年龄：20
学号：75， 姓名：Zhao， 性别：M， 年龄：20
学号：92， 姓名：Chen， 性别：M， 年龄：20
---顺序查找---
请输入要查找的学号(输入-1结束)： 75
学号：75， 姓名：Zhao， 性别：M， 年龄：20
查找时比较的次数=10

请输入要查找的学号(输入-1结束)： 100
当前要查找的记录不存在！
查找时比较的次数=11

请输入要查找的学号(输入-1结束)： -1

顺序查找成功的平均查找长度ASL=6.000000
顺序查找不成功的平均查找长度ASL=12.000000

```

```

学号有序的静态查找表
学号: 5, 姓名: Huang, 性别: M, 年龄: 20
学号: 13, 姓名: Qian, 性别: F, 年龄: 19
学号: 19, 姓名: Wang, 性别: F, 年龄: 20
学号: 21, 姓名: Zheng, 性别: M, 年龄: 20
学号: 37, 姓名: Sun, 性别: M, 年龄: 20
学号: 56, 姓名: Zhang, 性别: F, 年龄: 19
学号: 64, 姓名: Li, 性别: M, 年龄: 19
学号: 75, 姓名: Zhao, 性别: M, 年龄: 20
学号: 80, 姓名: Zhou, 性别: F, 年龄: 19
学号: 88, 姓名: Liu, 性别: F, 年龄: 18
学号: 92, 姓名: Chen, 性别: M, 年龄: 20
---折半查找---
请输入要查找的学号(输入-1结束): 21
学号: 21, 姓名: Zheng, 性别: M, 年龄: 20
查找时比较的次数=3

请输入要查找的学号(输入-1结束): 100
当前要查找的记录不存在!
查找时比较的次数=4

请输入要查找的学号(输入-1结束): -1

折半查找成功的平均查找长度ASL=3.000000
折半查找不成功的平均查找长度ASL=3.666667

```

实验 4-2 动态查找的实现

1. 实验内容

I. 动态查找表基本操作的实现

根据定义的存储结构，编写下列基本操作函数的 C/C++ 源代码：

(1) 编写一个插入函数 `InsertDSTable_BST(BSTree &DT, RecordType e)`，根据 BST 的性质将数据元素 `e` 插入到动态查找表 `DT` 中。若元素 `e` 在 `DT` 中已存在，则操作失败，返回 `FALSE`。

(2) 编写一个创建动态查找表的函数 `CreateDSTable_BST(BSTree &DT, SSTable &ST)`，其中 `ST` 为实验 4-1 所创建的静态查找表。该函数要调用 `InsertDSTable_BST()` 函数。

注：为提高程序调试速度，该函数允许直接用实验 4-1 所创建的静态查找表 `ST1` 中的数据元素。

(3) 编写一个中序遍历函数 `TraversDSTable_InOrde(BSTree DT)`，输出动态查找表 `DT` 的所有数据元素。

(4) 编写一个查找函数 `BSTree SearchDSTable_BST(BSTree DT, KeyType key)`，其中参数 `key` 为要查找元素的关键字。若查找成功，函数返回该关键字元素结点的地址（指针），否则，查找不成功，返回 `NULL`。该函数可以用递归或非递归方式实现，同时要计数查找过程中关键字比较的次数（递归函数可利用全局变量实现）。

(5) 编写一个删除函数 `DeleteDSTable_BST(BSTree &DT, KeyType key)`，其中参数 `key` 为要删除元素的关键字，若该关键字的元素在 `DT` 中不存在，则操作失败，返回 `FALSE`。

II. 编写一个主函数 `main()`，检验上述操作函数是否正确，实现以下操作：

(1) 定义一个静态查找表 `ST1`，将下列各元素的值写入 `ST1`，然后输出 `ST1` 的各元素的值。相关类型定义和操作函数可直接利用实验 4-1。

学号 key	姓名 name	性别 sex	年龄 age
56	Zhang	F	19
19	Wang	F	20
80	Zhou	F	19
5	Huang	M	20
21	Zheng	M	20
64	Li	M	19
88	Liu	F	18
13	Qian	F	19
37	Sun	M	20
75	Zhao	M	20
92	Chen	M	20

(2) 定义一个动态查找表 DT1，调用 CreateDSTable_BST(DT1,ST1)函数，根据静态查找表 ST1 中的数据元素创建动态查找表 DT1，然后调用 TraversDSTable_InOrder(DT1)函数输出 DT1 中的所有数据元素。

(3) 编写一段代码，根据输入的一个关键字，调用 SearchDSTable_BST()函数，查找动态表中的数据元素，若查找成功，输出找到的元素（记录）的值和关键字的比较次数，否则输出“当前要查找的记录不存在！”和关键字的比较次数。该段代码可以重复上述操作多次，关键字输入“-1”时结束。

(4) 依次使用静态查找表 ST1 中的关键字调用 SearchDSTable_BST()函数，查找动态表中的数据元素，计算并输出查找成功的 ASL。

(5) 选择使用不在静态查找表中的关键字进行查找，计算并输出查找不成功的 ASL。

(6) 编写一段代码，实现动态查找的元素插入操作。首先输入(scanf)要插入元素 e 的各个数据项的值，调用 InsertDSTable_BST(DT1, e)将元素 e 插入到动态查找表中，若插入成功，调用 TraversDSTable_InOrder(DT1)函数输出 DT1 中的所有数据元素，否则输出“当前要插入的记录已存在，禁止插入！”。该段代码可以重复上述操作多次，关键字输入“-1”时结束。

(7) 编写一段代码，实现动态查找的元素删除操作。首先输入要删除元素的关键字值，调用 DeleteDSTable_BST(DT1, key)将关键字为 key 的元素 e 从 DT1 中删除，若删除成功，调用 TraversDSTable_InOrder(DT1)函数输出 DT1 中的所有数据元素，否则输出“当前要删除的记录不存在，禁止删除！”。该段代码可以重复上述操作多次，关键字输入“-1”时结束。

2. 源程序代码 (*.CPP)

(见附件 dynamic_search.cpp)

3. 运行结果

```
静态查找表
学号: 56, 姓名: Zhang, 性别: F, 年龄: 19
学号: 19, 姓名: Wang, 性别: F, 年龄: 20
学号: 80, 姓名: Zhou, 性别: F, 年龄: 19
学号: 5, 姓名: Huang, 性别: M, 年龄: 20
学号: 21, 姓名: Zheng, 性别: M, 年龄: 20
学号: 64, 姓名: Li, 性别: M, 年龄: 19
学号: 88, 姓名: Liu, 性别: F, 年龄: 18
学号: 13, 姓名: Qian, 性别: F, 年龄: 19
学号: 37, 姓名: Sun, 性别: M, 年龄: 20
学号: 75, 姓名: Zhao, 性别: M, 年龄: 20
学号: 92, 姓名: Chen, 性别: M, 年龄: 20
```

```
动态查找表(中序遍历)
学号: 5 姓名: Huang 性别: M 年龄: 20
学号: 13 姓名: Qian 性别: F 年龄: 19
学号: 19 姓名: Wang 性别: F 年龄: 20
学号: 21 姓名: Zheng 性别: M 年龄: 20
学号: 37 姓名: Sun 性别: M 年龄: 20
学号: 56 姓名: Zhang 性别: F 年龄: 19
学号: 64 姓名: Li 性别: M 年龄: 19
学号: 75 姓名: Zhao 性别: M 年龄: 20
学号: 80 姓名: Zhou 性别: F 年龄: 19
学号: 88 姓名: Liu 性别: F 年龄: 18
学号: 92 姓名: Chen 性别: M 年龄: 20
```

---元素查找---

```
请输入要查找记录的学号(输入-1结束): 5
学号: 5, 姓名: Huang, 性别: M, 年龄: 20
查找时比较的次数=3
```

```
请输入要查找的学号(输入-1结束): 92
学号: 92, 姓名: Chen, 性别: M, 年龄: 20
查找时比较的次数=4
```

```
请输入要查找的学号(输入-1结束): 100
当前要查找的记录不存在!
查找时比较的次数=4
```

```
请输入要查找的学号(输入-1结束): -1
```

```
查找成功的平均查找长度ASL=3.000000
```

```
查找不成功的平均查找长度ASL=3.666667
```

---元素插入---

请输入要插入的新记录(输入-1结束)

学号 姓名 性别 年龄

学号: 6

姓名: hzh

性别: M

年龄: 21

插入元素后的动态查找表(中序遍历):

学号: 5 姓名: Huang 性别: M 年龄: 20

学号: 6 姓名: hzh 性别: M 年龄: 21

学号: 13 姓名: Qian 性别: F 年龄: 19

学号: 19 姓名: Wang 性别: F 年龄: 20

学号: 21 姓名: Zheng 性别: M 年龄: 20

学号: 37 姓名: Sun 性别: M 年龄: 20

学号: 56 姓名: Zhang 性别: F 年龄: 19

学号: 64 姓名: Li 性别: M 年龄: 19

学号: 75 姓名: Zhao 性别: M 年龄: 20

学号: 80 姓名: Zhou 性别: F 年龄: 19

学号: 88 姓名: Liu 性别: F 年龄: 18

学号: 92 姓名: Chen 性别: M 年龄: 20

学号: -1

---元素删除---

请输入要删除记录的学号(输入-1结束): 88

删除元素后的动态查找表(中序遍历):

学号: 5 姓名: Huang 性别: M 年龄: 20

学号: 6 姓名: 衍?性别: M 年龄: 21

学号: 13 姓名: Qian 性别: F 年龄: 19

学号: 19 姓名: Wang 性别: F 年龄: 20

学号: 21 姓名: Zheng 性别: M 年龄: 20

学号: 37 姓名: Sun 性别: M 年龄: 20

学号: 56 姓名: Zhang 性别: F 年龄: 19

学号: 64 姓名: Li 性别: M 年龄: 19

学号: 75 姓名: Zhao 性别: M 年龄: 20

学号: 80 姓名: Zhou 性别: F 年龄: 19

学号: 92 姓名: Chen 性别: M 年龄: 20

请输入要删除记录的学号(输入-1结束): -1

实验 4-3 散列表查找的实现

1. 实验内容

I. “线性探测再散列”哈希表基本操作的实现

(1) 编写一个创建哈希表的函数，函数格式如下：

`CreateHashTable(HashTable &HT, int HT_Length, KeyType key[], int KeyNum)`

函数参数说明如下：

`HashTable &HT`: 哈希表变量，存放哈希表的相关内容

`int HT_Length`: 哈希表的长度

`KeyType key[]`: 存放关键字的一维数组

`int KeyNum`: 关键字个数

该函数的哈希函数为 $\text{Hash}(\text{key}) = \text{key} \% 13$ ，以 $\text{Hash}(\text{key})$ 的值为下标将关键字存入哈希表 HT 中的 key 数组中，若有同义词，则采用“线性探测再散列”方法计算存放关键字的下标。没有存放关键字的数组元素用 -1 表示。

(2) 编写一个哈希查找函数，函数格式如下：

`SearchHashTable(HashTable HT, KeyType key, int &p, int &c)`

函数参数说明如下：

`HashTable HT`: 哈希表变量，存放哈希表的相关数据

`KeyType key`: 要查找的关键字

`int &p`: 若查找成功，表示关键字在哈希表中的下标，否则 $p = -1$

`int &c`: 关键字比较的次数

该函数根据参数 key 在哈希表中查找该关键字，若查找成功，返回 SUCCESS，否则返回 UNSUCCESS。

II. 链地址法”哈希表基本操作的实现

(1) 编写一个创建哈希表的函数，函数格式如下：

`CreateHashLink(HashLink &HL, int HL_Length, KeyType key[], int KeyNum)`

函数参数说明如下：

`HashLink &HL`: 哈希表变量，存放哈希表的相关数据

`int HL_Length`: 哈希表的长度（哈希函数“除留余数法”的除数）

`KeyType key[]`: 存放关键字的一维数组

`int KeyNum`: 关键字个数

该函数的哈希函数为 $\text{Hash}(\text{key}) = \text{key} \% 13$ ，同义词插入到以 $\text{Hash}(\text{key})$ 为下标的单链表中（采用头插法）。

(2) 编写一个哈希查找函数，函数格式如下：

`SearchHashLink(HashLink HL, KeyType key, KeyLink &p, int &c)`

函数参数说明如下：

`HashLink &HL`: 哈希表变量，存放哈希表的相关数据

`KeyType key`: 要查找的关键字

KeyLink &p: 若查找成功, 指向查找关键字结点的指针, 否则 p=NULL

int &c: 关键字比较的次数

该函数根据参数 key 在哈希表中查找该关键字在单链表中的结点, 若查找成功, 返回 SUCCESS, 否则返回 UNSUCCESS。

(3) 编写一个输出哈希表数据的函数 OutHashLink(HashLink HL), 输出格式如教材 P-225 图 7.30 所示。

III. 编写一个主函数 main(), 检验上述操作函数是否正确, 实现以下操作:

(1) 数据准备: 根据教材 p-224 例 7-2、p-227 例 7-3 给出的关键字, 定义一个一维数组 keys[] 存储这些关键字, 并输出这些关键字。

```
int keys[12]={19,14,23,1,68,20,84,27,55,11,10,79};
```

(2) 调用 CreateHashTable(HT,16, keys, 12) 函数创建一个“线性探测再散列”哈希表 HT, 然后输出哈希表的数据。(例 7.3)

(3) 依次使用 keys[] 数组中的关键字调用 SearchHashTable() 函数, 在哈希表 HT 中查找关键字, 输出各个关键字在哈希表中的位置(下标)和关键字的比较次数, 最后计算并输出“查找成功的 ASL”。(例 7.3)

(4) 调用 CreateHashLink(HL,13, keys, 12) 函数创建一个“链地址法”哈希表 HL, 然后调用 OutHashLink(HL) 函数输出哈希表 HL 的数据。(例 7.2)

(5) 依次使用 keys[] 数组中的关键字调用 SearchHashLink() 函数, 在哈希表 HL 中查找关键字, 输出各个关键字在查找时关键字的比较次数, 最后计算并输出查找成功的 ASL。(例 7.2)

(6) 定义一个一维数组 keys1[13] 如下:

```
keys1[13]={26,40,15,29,30,18,32,46,60,74,36,24,38};
```

其中的关键字都不在哈希表中, 取其中的关键字对哈希表 HT 和 HL 做查找操作, 输出两种查找时各个关键字的“关键字比较次数”, 然后计算并输出两种哈希表的“查找不成功的 ASL”。

2. 源程序代码 (*.CPP)

(见附件 hash_search.cpp)

3. 运行结果

```
关键字表:
19 14 23 1 68 20 84 27 55 11 10 79

线性探测再散列哈希表:
-1 14 1 68 27 55 19 20 84 79 23 11 10 -1 -1 -1

查找关键字=19 所在哈希表下标= 6 关键字比较次数= 1
查找关键字=14 所在哈希表下标= 1 关键字比较次数= 1
查找关键字=23 所在哈希表下标=10 关键字比较次数= 1
查找关键字= 1 所在哈希表下标= 2 关键字比较次数= 2
查找关键字=68 所在哈希表下标= 3 关键字比较次数= 1
查找关键字=20 所在哈希表下标= 7 关键字比较次数= 1
查找关键字=84 所在哈希表下标= 8 关键字比较次数= 3
查找关键字=27 所在哈希表下标= 4 关键字比较次数= 4
查找关键字=55 所在哈希表下标= 5 关键字比较次数= 3
查找关键字=11 所在哈希表下标=11 关键字比较次数= 1
查找关键字=10 所在哈希表下标=12 关键字比较次数= 3
查找关键字=79 所在哈希表下标= 9 关键字比较次数= 9

查找成功 ASL=2.500000
```

```
查找关键字=26 所在哈希表下标=-1 关键字比较次数= 1
查找关键字=40 所在哈希表下标=-1 关键字比较次数=13
查找关键字=15 所在哈希表下标=-1 关键字比较次数=12
查找关键字=29 所在哈希表下标=-1 关键字比较次数=11
查找关键字=30 所在哈希表下标=-1 关键字比较次数=10
查找关键字=18 所在哈希表下标=-1 关键字比较次数= 9
查找关键字=32 所在哈希表下标=-1 关键字比较次数= 8
查找关键字=46 所在哈希表下标=-1 关键字比较次数= 7
查找关键字=60 所在哈希表下标=-1 关键字比较次数= 6
查找关键字=74 所在哈希表下标=-1 关键字比较次数= 5
查找关键字=36 所在哈希表下标=-1 关键字比较次数= 4
查找关键字=24 所在哈希表下标=-1 关键字比较次数= 3
查找关键字=38 所在哈希表下标=-1 关键字比较次数= 2
```

查找不成功ASL=7.000000

链地址法哈希表：

```
Hash[0]: ^
Hash[1]: 79 -> 27 -> 1 -> 14 -> ^
Hash[2]: ^
Hash[3]: 55 -> 68 -> ^
Hash[4]: ^
Hash[5]: ^
Hash[6]: 84 -> 19 -> ^
Hash[7]: 20 -> ^
Hash[8]: ^
Hash[9]: ^
Hash[10]: 10 -> 23 -> ^
Hash[11]: 11 -> ^
Hash[12]: ^
```

```
查找关键字=19 所在哈希表下标= 6 关键字比较次数= 2
查找关键字=14 所在哈希表下标= 1 关键字比较次数= 4
查找关键字=23 所在哈希表下标=10 关键字比较次数= 2
查找关键字= 1 所在哈希表下标= 1 关键字比较次数= 3
查找关键字=68 所在哈希表下标= 3 关键字比较次数= 2
查找关键字=20 所在哈希表下标= 7 关键字比较次数= 1
查找关键字=84 所在哈希表下标= 6 关键字比较次数= 1
查找关键字=27 所在哈希表下标= 1 关键字比较次数= 2
查找关键字=55 所在哈希表下标= 3 关键字比较次数= 1
查找关键字=11 所在哈希表下标=11 关键字比较次数= 1
查找关键字=10 所在哈希表下标=10 关键字比较次数= 1
查找关键字=79 所在哈希表下标= 1 关键字比较次数= 1
```

查找成功ASL=1.750000

```
查找关键字=26 所在哈希表下标= 0 关键字比较次数= 1
查找关键字=40 所在哈希表下标= 1 关键字比较次数= 5
查找关键字=15 所在哈希表下标= 2 关键字比较次数= 1
查找关键字=29 所在哈希表下标= 3 关键字比较次数= 3
查找关键字=30 所在哈希表下标= 4 关键字比较次数= 1
查找关键字=18 所在哈希表下标= 5 关键字比较次数= 1
查找关键字=32 所在哈希表下标= 6 关键字比较次数= 3
查找关键字=46 所在哈希表下标= 7 关键字比较次数= 2
查找关键字=60 所在哈希表下标= 8 关键字比较次数= 1
查找关键字=74 所在哈希表下标= 9 关键字比较次数= 1
查找关键字=36 所在哈希表下标=10 关键字比较次数= 3
查找关键字=24 所在哈希表下标=11 关键字比较次数= 2
查找关键字=38 所在哈希表下标=12 关键字比较次数= 1
```

查找不成功ASL=1.923077

三、实验小结

（通过实验得出的结论，对算法的功能以及与程序之间区别的理解、在编码是碰到的问题以及解决办法等心得以及得到的收获）

通过静态查找表、动态查找表（BST）和哈希表查找的实验，我对三种常见查找算法有了更深入的理解。静态查找表虽然实现简单，但其查找效率较低，特别是在数据量大时，查找性能明显下降，适合静态、查询频繁的场景。动态查找表（BST）利用二叉树结构，提供了较好的查找效率，平衡树能够保持 $O(\log n)$ 的查找时间复杂度，但如果树不平衡，最坏情况下会退化为链表，查找效率急剧下降。相比之下，哈希表通过哈希函数实现 $O(1)$ 的理想查找效率，特别适用于高频查找场景，但哈希冲突的处理成为其性能瓶颈，线性探测和链地址法各有优劣。

在编码过程中，我遇到了多个挑战，特别是在链表和哈希表的内存管理和指针操作上。链表的插入和查找需要注意指针的正确链接，避免内存泄漏或野指针。哈希表的冲突处理则让我深入思考了如何平衡查找效率和内存开销，线性探测虽然简单，但容易产生聚集，而链地址法则在冲突较多时会导致链表过长，影响查找效率。通过不断调试和优化，我逐渐掌握了指针的使用技巧和内存管理的方法，尤其是在处理动态内存分配和释放时需要格外小心。

这些实验不仅让我理解了不同数据结构和算法的理论基础，更让我深刻体会到实际编码时的复杂性。选择合适的数据结构与算法，结合具体应用场景，能够显著提高程序的性能和可维护性。这些经验不仅丰富了我的编程技巧，也帮助我在后续的学习和项目中能够更高效地处理各种数据查找问题。